

BillSplit

Architecture & Full Project Flow

Interview preparation guide — what is already built and what comes next

1. What the App Is

BillSplit is a smart restaurant bill-splitting Android app built with Flutter. The user photographs a restaurant bill; on-device Google ML Kit OCR extracts the text; the app parses it into items and prices; the user edits the detected items, assigns each item to the friends who shared it, and BillSplit calculates each person's share including a proportional split of the tax. Finally it generates UPI payment links that can be shared with friends. Friends and bill history are stored per-user in Firebase Cloud Firestore.

The one-line pitch: "Scan the bill. Split it fairly. Settle instantly."

End-to-end user journey

```
Splash (brand animation)
→ AuthGate (listens to Firebase auth state)
  → LoginScreen (email/password + confirm password, Google Sign-In)
  → HomeScreen (dashboard: searchable live friends list)
    → FriendsScreen (manage friends: add / delete)
    → [planned] ScanScreen → EditItemsScreen → AssignScreen
      → ResultsScreen (per-person totals + UPI links + share)
      → saved to Firestore → HistoryScreen
```

2. Technology Stack (and why)

Technology	Role	Why this choice
Flutter + Dart	Cross-platform UI (Android, minSdk 21)	Single codebase, fast development, Material 3 built in
Material 3	Design system	Modern Android look; theming via <code>ColorScheme.fromSeed</code>
Provider	State management	Lightweight, fits the app's small shared-state needs; no Bloc/Riverpod boilerplate
Firebase Auth + Google Sign-In	Authentication	Managed, secure auth; no backend to build
Cloud Firestore	Database	Realtime streams, offline cache, per-user security rules
Google ML Kit Text Recognition	OCR (planned)	On-device: free, fast, works offline, bill photos never leave the phone
image_picker	Camera/gallery (planned)	Standard plugin for photo input
url_launcher + share_plus	UPI links (planned)	Open <code>upi://</code> deep links and share via any app
uuid, intl	IDs and formatting	Stable document ids; Indian currency formatting (₹)

3. Architecture: Feature-First Clean Architecture

The codebase is organised **by feature** (auth, friends, home, splash, scan, split, history) and inside each feature **by layer**. Cross-cutting code lives in **core/** (models, theme, utils, constants) and **shared/** (reusable widgets).

```
lib/  
  main.dart                # Firebase.initializeApp + runApp  
  firebase_options.dart    # generated by flutterfire configure  
  app/app.dart             # BillSplitApp: MultiProvider + MaterialApp  
  core/  
    constants/             app_const.dart, app_images.dart  
    models/                friend, bill_item, settlement, bill    (pure Dart)  
    theme/                 app_colors.dart, app_theme.dart        (Material 3)  
    utils/                 validation.dart (+ planned currency_formatter, upi_link_builder)  
  shared/widgets/           app_button, app_text_field, app_logo  
  features/  
    splash/                presentation/screens/splash_screen.dart  
    auth/                  data/services/auth_service.dart  
                           presentation/providers/auth_provider.dart  
                           presentation/screens/auth_gate.dart, login_screen.dart  
    home/                  presentation/screens/home_screen.dart    (dashboard)  
    friends/               data/repositories/friends_repository.dart  
                           presentation/screens/friends_screen.dart  
                           presentation/widgets/add_friend_dialog, friend_card  
    scan/                  [planned] ocr_service, bill_parser, scan/edit screens  
    split/                 [planned] settlement_calculator, bill_flow_state, assign/results screens  
    history/               [planned] bills_repository, history_screen
```

Layer responsibilities (memorise this)

Layer	Contains	Rule
Presentation	Screens, widgets, provider consumption	Never touches Firestore or FirebaseAuth directly
State	StreamProvider, ValueNotifier, (planned) BillFlowState ChangeNotifier	Only where UI actually needs shared state
Data	AuthService, FriendsRepository, (planned) BillsRepository, OcrService	Repositories are the ONLY classes that access Firestore
Domain	bill_parser, settlement_calculator, upi_link_builder, models	Pure Dart: no Flutter, no Firebase imports; fully unit-testable

4. State Management Decisions

- **StreamProvider<User?>** (auth_provider.dart) exposes `FirebaseAuth.authStateChanges()` to the whole widget tree. AuthGate simply watches it: `user == null` → LoginScreen, else HomeScreen. Signing in or out swaps the screen automatically — no manual navigation.
- **Provider<AuthService>** gives screens access to auth actions (sign in, register, reset, Google, sign out) without constructing Firebase objects in the UI. In widget tests a FakeAuthService is injected instead, so tests never touch Firebase.
- **Plain StreamBuilder** for Firestore lists (friends on the dashboard and friends screen). Firestore snapshots are already a stream — wrapping them in another state class would add nothing.
- **ValueNotifier + ValueListenableBuilder** for screen-local UI state: password visibility, sign-in vs sign-up mode, submit spinner, dashboard search query. Only the widget that depends on the value rebuilds — more efficient and more intentional than `setState` over the whole screen.
- **[Planned] ChangeNotifierProvider<BillFlowState>** for the temporary Scan → Edit → Assign → Settle flow, because multiple screens must share one evolving draft bill.

Why not Bloc/Riverpod? The app has exactly two kinds of shared state (auth user, draft bill) plus streams Firestore already provides. Provider covers this with minimal boilerplate; Bloc would add events/states/mappers with no benefit at this scale. This restraint — not creating a ChangeNotifier per screen — is a deliberate architectural decision.

5. Authentication Flow (fully implemented)

Startup

- `main()` → `WidgetsFlutterBinding.ensureInitialized()` → `Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform)` → `runApp(BillSplitApp)`.
- `BillSplitApp` wraps `MaterialApp` in `MultiProvider` with `Provider<AuthService>` and `StreamProvider<User?>` (`initialData: currentUser`, so an already-signed-in user never flashes the login screen).
- `SplashScreen` plays a staggered intro (logo fade+scale, tagline fade+slide via one `AnimationController` with `Intervals`), then after `AppConstants.splashMinDuration` fades to `AuthGate`.

Email / password

- LoginScreen validates with the shared **Validators** utility (core/utils/validation.dart): email regex; password min 8 chars with at least one letter and one digit; sign-up mode adds a **Confirm Password** field that must match.
- **AuthService** wraps FirebaseAuth and is the only class that talks to it. FirebaseAuthException codes are mapped to friendly messages (wrong password, email already in use, weak password, network error...) and rethrown as a small `AuthException` the UI shows in a snackbar.
- Forgot Password validates the email field, then sends FirebaseAuth's reset email.

Google Sign-In (google_sign_in v7)

```
GoogleSignIn.initialize(serverClientId: <web client id>)  
  → authenticate() # Android Credential Manager account picker  
  → account.authentication.idToken  
  → GoogleAuthProvider.credential(idToken)  
  → FirebaseAuth.signInWithCredential(credential)
```

- The **serverClientId** is the OAuth **web** client id generated when the Google provider was enabled in the Firebase console; the debug keystore **SHA-1** is registered with the Firebase Android app so Google can verify the app's signature. Cancelling the picker is treated as a non-error.
- On success nothing navigates manually: the auth stream emits the new User, AuthGate rebuilds and shows HomeScreen. Sign-out is the same mechanism in reverse.

6. Firestore Data Model

```
users/{uid}/friends/{friendId} # implemented  
  name: string, upiId?: string, phone?: string  
  
users/{uid}/bills/{billId} # planned (history feature)  
  restaurantName, createdAtMillis, taxAmount, totalAmount,  
  items: [{id, name, price, sharedByFriendIds[]}],  
  settlements: [{friendId, friendName, itemsTotal, taxShare}]
```

- **Why per-user subcollections?** Each user's data is naturally isolated under their uid — security rules become one line (`request.auth.uid == uid`), queries never need owner filtering, and there is no shared-collection contention.
- **Live lists:** `FriendsRepository.watchFriends()` maps `collection.orderBy('name').snapshots()` to `List<Friend>`. Any write — even from another device — pushes a new list into the StreamBuilder and the UI updates instantly. Firestore's local cache makes writes feel instant (latency compensation) and gives basic offline support for free.
- **Current rules are test mode** (open, 30-day expiry) — acceptable during development; production rules would restrict each `users/{uid}` subtree to `request.auth.uid == uid`.

Domain models (core/models/ — pure Dart)

- **Friend**(id, name, upiId?, phone?) — id is the Firestore doc id, kept out of the map.
- **BillItem**(id, name, price, sharedByFriendIds[]) — the assignment of items to people.
- **Settlement**(friendId, friendName, itemsTotal, taxShare) with computed `totalOwed`.
- **Bill**(id, restaurantName, createdAt, items[], taxAmount, totalAmount, settlements[]).

- **createdAt is stored as milliseconds since epoch**, not a Firestore Timestamp — that keeps the models free of Firebase imports so they are trivially unit-testable (test/models_test.dart does toMap/fromMap round-trips). Type conversion is the repository's job.

7. What Is DONE vs. What Comes NEXT

Done (visible in commit history)

- Project scaffold with the full feature-first folder structure (committed first — shows the architecture was designed up front).
- Material 3 theme from brand colours (blue #075CE6, teal #00CDA4, navy #011B44), Outfit font, shared AppButton / AppTextField / AppLogo widgets.
- Animated splash screen with brand artwork.
- Login screen matching the design mock: email+password with validation, confirm-password on sign-up, forgot password, Google button with a canvas-painted "G" logo (no asset needed).
- Firebase wiring: flutterfire configure, google-services.json, SHA-1 registration, Firebase.initializeApp.
- Full authentication: AuthService, StreamProvider auth state, AuthGate, friendly error mapping, ValueNotifier-driven UI state.
- Core domain models + round-trip unit tests.
- Friends feature: FriendsRepository (the only Firestore access point), add-friend dialog with Validators (UPI id, Indian mobile), live list with delete confirmation.
- Home dashboard: greeting, searchable live friends list (filter by name/UPI/phone), add-friend FAB, "See all" navigation.
- 12 automated tests: 6 model unit tests + 6 widget tests (widget tests inject FakeAuthService so no Firebase is needed).

Next (in build order)

- **Scan feature:** image_picker (camera/gallery) → OcrService wrapping ML Kit TextRecognizer.processImage → raw text.
- **bill_parser.dart (pure Dart):** line-by-line parsing of OCR text into BillItem candidates — match "name ... price" patterns, detect tax/GST lines and the total, ignore noise. Unit-tested against sample bill texts (test/bill_parser_test.dart).
- **EditItemsScreen:** fix OCR mistakes — rename, re-price, add/remove items, set tax.
- **BillFlowState (ChangeNotifier):** holds the draft bill through Scan → Edit → Assign → Results; discarded when the flow completes.
- **AssignScreen:** for each item pick which friends shared it (chips/checkboxes).
- **settlement_calculator.dart (pure Dart):** per item, `price / sharedBy.length` is added to each sharer; tax is distributed proportionally: `taxShare_i = tax × itemsTotal_i / itemsSubtotal`. Unit-tested including rounding edge cases.
- **Worked example:** items ₹300 (A) + ₹100 (A+B, ₹50 each) → A items ₹350, B items ₹50; tax ₹40 on subtotal ₹400 → A pays 350 + 35 = **₹385**, B pays 50 + 5 = **₹55**.
- **ResultsScreen + upi_link_builder.dart (pure Dart):** per friend build `upi://pay?pa=<upiId>&pn=<name>&am=<amount>&cu=INR`, opened with url_launcher or shared with share_plus.
- **History:** BillsRepository saves the finished Bill to `users/{uid}/bills`; HistoryScreen lists past bills with a StreamBuilder, newest first.

- **Hardening:** production Firestore security rules, currency formatting with intl, release signing config + its SHA-1.

8. Likely Interview Questions — with Answers

Q. Why did you choose Provider over Bloc or Riverpod?

The app's shared state is small: the authenticated user (already a stream from Firebase) and one draft bill during the split flow. Provider exposes both with almost no boilerplate. Bloc's event/state classes would triple the code with no testability gain here — my business logic is already isolated in pure-Dart classes that need no framework to test. I also deliberately avoided creating a ChangeNotifier per screen; screen-local state uses ValueNotifier.

Q. Walk me through what happens when the app starts.

main() initialises Firebase with the generated options, then runs BillSplitApp. That widget creates AuthService and registers two providers: Provider<AuthService> and StreamProvider<User?> fed by authStateChanges() with currentUser as initialData. MaterialApp shows the splash, which plays a 1.4-second staggered animation and then replaces itself with AuthGate. AuthGate watches the User? provider: null shows LoginScreen, non-null shows HomeScreen.

Q. How does login actually authenticate?

The form validates locally first (shared Validators). Then LoginScreen calls AuthService — the only class that touches FirebaseAuth. For email it calls signInWithEmailAndPassword or createUserWithEmailAndPassword; for Google it runs the google_sign_in v7 flow: initialize with the server client id, authenticate() opens the Credential Manager picker, and the returned idToken is exchanged for a Firebase credential via signInWithCredential. Errors are mapped from FirebaseAuthException codes to friendly messages. On success no navigation code runs — the auth stream emits and AuthGate rebuilds.

Q. Why does Google Sign-In need a SHA-1 and a "server client id"?

Google verifies the calling Android app by its package name + signing-certificate SHA-1, which must be registered in Firebase. The serverClientId is the project's OAuth web client id; Credential Manager needs it to mint an ID token that Firebase (the "server") can verify.

Q. Why are bill_parser, settlement_calculator and upi_link_builder pure Dart?

They are the core business logic. Keeping them free of Flutter and Firebase imports means they run in plain dart tests — no emulators, no mocking frameworks — and they could be reused on any platform. It's the Dependency Rule from Clean Architecture: the domain depends on nothing.

Q. Why can only repositories access Firestore?

Single responsibility and swappability. Screens declare what they need (a stream of friends); the repository decides where it comes from. If I migrated the backend or added caching, only the repository changes. It also keeps the security-review surface tiny.

Q. How is the tax split fairly?

Proportionally to consumption, not per head: each person's taxShare = totalTax × theirItemsTotal ÷ itemsSubtotal. Someone who ordered 70% of the food pays 70% of the GST. An equal per-head tax would punish light eaters.

Q. How do your widget tests run without Firebase?

BillSplitApp accepts an optional AuthService. Tests pass a FakeAuthService whose authStateChanges() returns Stream.value(null), so the provider tree works and no Firebase initialisation is needed. That's constructor injection — the same seam I'd use to mock repositories later.

Q. Why on-device ML Kit OCR instead of a cloud OCR API?

Free, no network needed in a restaurant, low latency, and privacy — the bill photo never leaves the device. Accuracy on printed receipts is good, and the Edit screen exists precisely because OCR is never perfect.

Q. How does the friends list update in real time?

FriendsRepository.watchFriends() returns Firestore's snapshots() stream mapped into model objects; StreamBuilder rebuilds on every emission. Writes hit Firestore's local cache first (latency compensation), so the UI updates instantly even before the server confirms.

Q. What about Firestore security?

Data is namespaced as users/{uid}/... so production rules are one condition: allow read, write on users/{uid}/{document=**} if request.auth != null && request.auth.uid == uid. The project currently runs test-mode rules during development.

Q. Why store createdAt as milliseconds instead of a Firestore Timestamp?

So the Bill model has zero Firebase imports and stays pure, unit-testable Dart. The repository layer owns type conversion. millisecondsSinceEpoch is unambiguous and sortable.

Q. Where does the confirm-password check happen?

Client-side, in the shared Validators.confirmPassword, wired into the Confirm field's validator only in sign-up mode. Firebase only ever receives one password; confirmation is purely a UX guard against typos.

Q. How would you scale this — groups, bills shared between users?

Move bills to a top-level collection with a memberUids array, secure with "request.auth.uid in resource.data.memberUids", add an invite flow, and use Cloud Functions for cross-user notifications. Because of the repository abstraction, screens wouldn't change.

Q. What was the hardest bug you hit?

Two good stories. (1) UnimplementedError / channel-error after adding Firebase plugins — native plugin code cannot be injected by hot restart; it needs a full rebuild. That explains how Flutter plugins bridge to platform code. (2) CONFIGURATION_NOT_FOUND on sign-up — the server-side Authentication product was never initialised in the console; it taught me the difference between client config (google-services.json) and server-side project state.